

Slay the Roadmap / Slay the Code

GamiDOC di Progetto

D'Agostino (matr. 303226)

Di Giacomo (matr. 303377)

Corso di Esempi di Gamification e Sistemi Intelligenti (EGS)

A.A. 2025/2026 — 9 settembre 2026

Repository: https://github.com/Ago95Dev/slay_the_roadmap

*Nota sui nomi: progetto **Slay the Roadmap**, game Hub **Slay The Code** (game su console Hub).*

Indice

1	Context	3
1.1	Concept	3
1.2	Obiettivi didattici	3
1.3	Target users	3
1.4	Perché faticano (contesto del problema)	3
1.5	Domain	4
1.6	Comportamenti incoraggiati	4
1.7	Comportamenti scoraggiati e anti-abuso	4
1.8	Tecnologia	4
1.9	Fasi e durate tipiche	4
1.10	Struttura della campagna	5
2	Game behavior	6
2.1	Core loop	6
2.2	Boss fight	6
2.3	Regole di gioco	7
2.4	Meccaniche delle carte	7
2.5	Progressione	7
2.6	Economie numeriche locali	7
3	Gamification	9
3.1	Meccaniche e dinamiche (Molina/Toda)	9
3.2	Lente Octalysis	9
4	Aesthetics e Dynamics	11
4.1	Struttura dei capitoli (esempio)	11
4.2	Carte evolutive (esempio)	11
4.3	Dinamiche positive	12
4.4	Dinamiche negative e mitigazioni	12
4.5	Personalizzazione	12
4.6	Adattabilità	12
5	Architettura e scelte tecniche	13
5.1	Layer	13
5.2	Integrazione Hub (contratto)	14
5.3	Scelte tecniche	14
6	Stato implementazione (onesto)	15
6.1	Cosa gira davvero	15
6.2	Tagli dichiarati (future work)	15

7	Valutazione, KPI e Analytics	16
7.1	Valutazione (protocollo da eseguire)	16
7.2	KPI e Analytics	16

Capitolo 1

Context

1.1 Concept

Slay the Roadmap / Slay the Code è un dungeon crawler didattico offline-first realizzato in Flutter. La **roadmap** è il **dungeon**, i **topic** sono i **mostri/stanze**, le **carte-ricompensa** sono il **bottino**. Il giocatore studia un topic, supera un quiz e avanza lungo la roadmap fino al boss di capitolo. La campagna implementata è il **track Dart/Flutter** (dati live in `lib/data/roadmap_data.dart` e `lib/data/topics_and_quizzes.dart`): fondamentali Dart (tipi, operatori, null safety, controllo di flusso, funzioni, collezioni, OOP, async) e basi Flutter (widget, layout, state management), quiz in italiano e inglese tecnico.

1.2 Obiettivi didattici

- Costruire i modelli mentali di Dart/Flutter prima della sintassi avanzata: tipi e null safety, flusso di controllo, dove vive lo stato, cosa fa una chiamata async.
- Dare feedback immediato e motivante: ogni quiz superato sblocca il nodo successivo e assegna una ricompensa; ogni capitolo si chiude con un boss.
- Rendere misurabile il progresso sia in locale (XP, livelli, vite, streak, save) sia sull'Hub (XP, badge, livelli remoti).

1.3 Target users

Studenti al primo approccio a Dart/Flutter (tipicamente primo anno di indirizzo informatico, 18–22 anni), con basi deboli o nulle di OOP, programmazione asincrona o widget: i testi dei topic partono dalle idee e i link di approfondimento (documentazione Dart/Flutter, pub.dev) sono opzionali. Esclusi di proposito: sviluppatori esperti (nessuna sfida) e pubblico generalista (contenuti troppo specifici). Il gioco è pensato per sessioni brevi (pochi minuti per topic) e per essere rigiocabile da zero (New Run).

1.4 Perché faticano (contesto del problema)

Chi inizia con Dart/Flutter parte di solito copiando widget e snippet senza modelli mentali: tipi e null safety, dove vive lo stato, cosa fa davvero una chiamata async. Il risultato è incollare codice senza capirlo e bloccarsi al primo errore. La campagna inverte l'ordine: prima tipi/flusso/stato/async (perché/come/cosa-succede-se), poi la costruzione di UI, con i boss come prove sommative.

1.5 Domain

Ambito applicativo: sviluppo mobile multiplatforma con Flutter/Dart, con allineamento agli standard di industria (OOP, async, widget e layout come orizzonte, non come prerequisito). La campagna Dart/Flutter copre tipi e operatori, flusso di controllo, dati e stato, OOP e async, widget e layout: gli stessi assi su cui un junior Flutter viene valutato nel mercato del lavoro.

1.6 Comportamenti incoraggiati

- **Costanza:** sessioni brevi e quotidiane (daily reward) invece di maratone;
- **Mastery concettuale:** soglia 80 % al quiz, ripasso dei topic falliti, hint dopo errori ripetuti;
- **Pensiero strategico:** scelta 1-di-3 e ottimizzazione del deck in funzione del boss;
- **Problem solving adattivo:** quiz del boss pescati dai topic del capitolo, passive diverse per boss.

1.7 Comportamenti scoraggiati e anti-abuso

- **Spam di tentativi:** i retry non danno XP (quiz) o danno retry senza XP (boss); lo skip-che-sbloccava è stato rimosso;
- **Doppie ricompense:** guard 1/topic con messaggio, anche via re-claim;
- **Farming XP:** XP solo ad azioni compiute (quiz +100, boss +100, topic +50, risorsa +30, dungeon +200, daily +25 una-tantum); nessun XP per sconfitte o login ripetuti;
- **Override manuale:** la voce “Done” resta come scelta esplicita dichiarata, non come scorciatoia nascosta.

Il sistema non punisce mai: nessuna perdita di progressi, carte o livelli.

1.8 Tecnologia

Flutter con un'unica codebase (sviluppo e verifica su Linux desktop, stessa logica su mobile/web): manutenibilità e portabilità senza backend proprio. Persistenza offline-first su SharedPreferences (save per utente); Gamification Hub esterno usato solo come specchio di XP, badge e livelli con fallback totale (dettagli al Cap. 5).

1.9 Fasi e durate tipiche

Fase	Durata	Attività	Obiettivo
Studio	6–8 min	Lettura topic	Acquisizione modelli mentali
Quiz	3–5 min	Verifica 5Q	Comprensione (soglia 80 %)
Deck building	2–3 min	Scelta 1-di-3	Scelte motivanti
Boss fight	10–15 min	Applicazione	Prova sommativa

Tabella 1.1: Durate indicative per fase di gioco.

Sequenza: selezione topic → studio → quiz → ricompensa + XP → costruzione e ottimizzazione deck → sfida boss → ricompensa e sblocco del capitolo successivo, in modo iterativo.

1.10 Struttura della campagna

Quattro capitoli da 3–4 topic quiz ciascuno, con nodi extra (tesori, eventi, riposo, elite) e 1 boss finale per capitolo, per un totale di 29 nodi e 11 quiz. Tutti gli ID sono stabili in snake_case (Tab. 1.2, schema in Fig. 1).

Capitolo	Topic quiz	Boss
1 – Dart Basics	Variables & Types, Operators, Null Safety	<code>syntax_sentinel</code>
2 – Control & Data	Conditionals, Loops, Functions, Collections	<code>logic_leviathan</code>
3 – OOP & Async	Classes & Objects, Inheritance & Mixins, Async & Futures	<code>abstraction_archon</code>
4 – Flutter Basics	Widgets Basics, Layouts, State Management Basics	<code>widget_warlord</code>

Tabella 1.2: Mappa stabile della campagna Dart/Flutter.

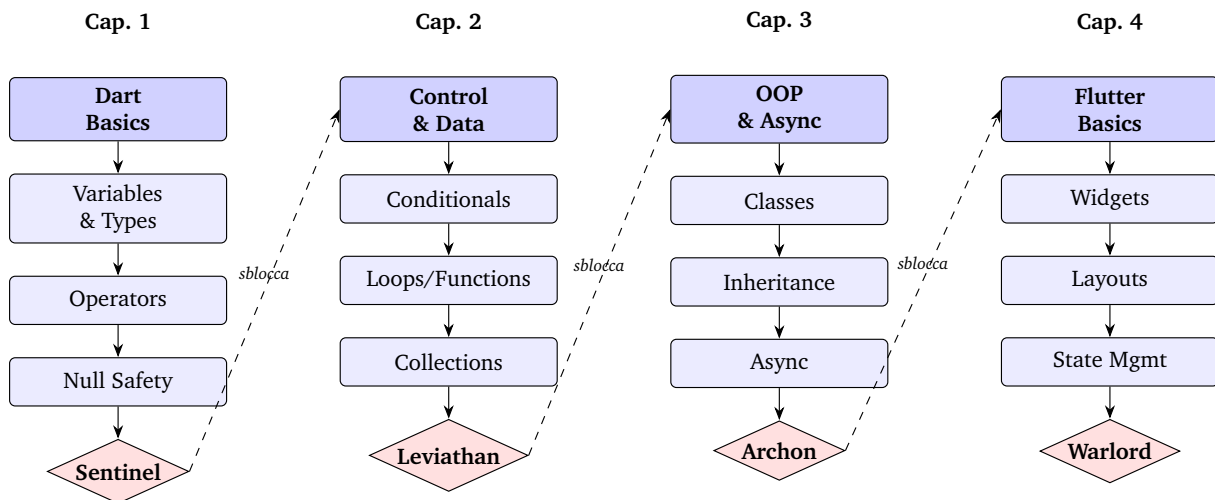


Figura 1: Albero della roadmap: 4 capitoli × 3–4 topic quiz + boss di capitolo. La sconfitta del boss (Sentinel = `syntax_sentinel`, Leviathan = `logic_leviathan`, Archon = `abstraction_archon`, Warlord = `widget_warlord`) sblocca il capitolo successivo.

Prerequisiti a catena: ogni capitolo richiede la sconfitta del boss del precedente (Sentinel → Leviathan → Archon → Warlord); gli ID completi sono in Tab. 1.2. Ogni topic quiz contiene 5 domande con soglia 80 %.

Capitolo 2

Game behavior

2.1 Core loop

Il loop (riassunto in Fig. 2) procede così:

1. **Studio:** pagina di dettaglio del topic (testo breve + link opzionali). Nessun timer.
2. **Quiz:** 5 domande a risposta multipla, soglia di superamento **80 % (4/5)**. I tentativi sono illimitati ma penalizzati (nessuna XP in caso di sconfitta/ripetizione fallita, perdita di vite nel boss).
3. **Ricompensa:** a quiz superato, scelta **1-di-3** carte (per tipo di ricompensa). Una sola ricompensa per topic; inventario visibile.
4. **Avanzamento:** lo sblocco del nodo successivo lungo la roadmap; a fine capitolo, **boss fight**.

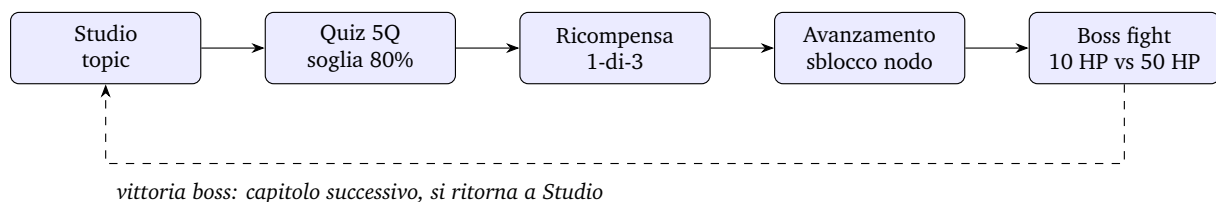


Figura 2: Core loop: Studio → Quiz → Ricompensa → Avanzamento → Boss fight, con ritorno al capitolo successivo.

2.2 Boss fight

Combattimento a turni con numeri reali (stato vero implementato):

- **Boss: 10 HP** (clamp dai valori narrativi 80/120/180/200); **giocatore: 50 HP** da Player Stats; **vite totali 3**.
- **Energia 3 per turno** (1 per carta giocata).
- A fine turno quiz obbligatorio: risposta giusta = **−10 al boss** (−20 sulle soglie: chiude il fight in un colpo, dichiarato come semplificazione); risposta errata = danno intero della mossa boss (8–30, mitigabile con Block).

- Vittoria: claim delle ricompense del nodo boss (semplificazione dichiarata: tutte quelle del nodo, non 1 sola), sblocco del capitolo successivo, badge del boss, **+100 XP** (come l'Hub). Sconfitta: retry a HP pieni, senza XP.

I quattro boss (con mosse e soglie reali dai dati):

- **Syntax Sentinel** (cap. 1): *Type Error* (8 danni + Vulnerable); sotto 50 % HP +2 Strength, sotto 25 % danni doppi;
- **Logic Leviathan** (cap. 2): *Infinite Loop* (10 danni + Bleed); sotto 60 % due abilità per turno, sotto 30 % Poison a inizio turno;
- **Abstraction Archon** (cap. 3): *Polymorphic Blast* e *Override* (18 danni, ignora Block); soglie 75/50/25 (copia carta, immunità status, +10 danni);
- **Widget Warlord** (cap. 4): *SetState Crash* e *Expanded Flex* (25 danni + Vulnerable); il fight usa HP 10 contro 3 come gli altri.

I quiz del boss sono pescati dai topic del capitolo.

2.3 Regole di gioco

- Sblocco progressivo con prerequisiti a catena (nodo $n+1$ richiede il nodo n ; capitolo $k+1$ richiede il boss del capitolo k).
- Quiz da 5 domande, soglia **80 %**, tentativi illimitati ma penalizzati (nessuna XP se non superato).
- Boss: 10 HP contro 50 HP, energia 3 per turno (1 per carta), 3 vite totali; quiz-danno a fine turno.
- Una sola ricompensa per topic (scelta 1-di-3); retry boss a HP pieni senza XP.

2.4 Meccaniche delle carte

21 carte in 4 tipi (attack, defense, utility, knowledge) e 4 rarità (7 comuni, 5 rare, 5 epiche, 4 leggendarie), con costo in energia da 1 a 4 e status (poison, burn, bleed, vulnerable, ...): es. *Strike* (6 danni, costo 1), *Apocalypse* (25 danni + status, costo 4). Le carte si ottengono solo come ricompensa 1-di-3 e compongono deck e inventario.

2.5 Progressione

3 capitoli $\times$ (root + 3 topic) + boss finale come prova sommativa; XP e livelli con reward scalari (quiz +100, boss +100; soglie 0/100/500 coincidenti con l'Hub); badge di topic e di boss; titoli di capitolo e avatar come marcatori di avanzamento.

2.6 Economie numeriche locali

- **XP/livelli**: soglie **L1 0 / L2 100 / L3 500**, identiche a quelle dell'Hub così che locale e remoto coincidano. Quiz superato: **+100 XP**; vittoria boss: **+100 XP** (stessi dell'Hub).
- **Streak**: conteggiata ai fini del ripasso (failCount, soglia di ripasso); il bonus XP è definito ma **non attivo** e non va considerato nella valutazione.

- **Vite: 3. Save per utente:** SharedPreferences (Save v1: nodi completati, deck, XP, livello, boss sbloccati, reward riscattate, badge; Continue ripristina, New Run chiede conferma e azzera solo quell'utente). Auth locale con hash salato e migrazione dalle password storiche.
- **Animazioni:** sobrie (transizioni essenziali, niente effetti extra).

Capitolo 3

Gamification

3.1 Meccaniche e dinamiche (Molina/Toda)

Gli elementi implementati, classificati secondo la tassonomia di Toda et al. [4]:

- **Performance:** punti XP (quiz +100, boss +100, topic +50, risorsa +30, dungeon +200, daily +25), livelli (0/100/500), streak conteggiata (bonus non attivo), soglia quiz 80 %, feedback immediato con explanation come hint dopo errori ripetuti.
- **Ecological:** vite (3), energia boss (3/turno), economia chiusa senza valuta acquistabile.
- **Personal:** collezione carte (deck da ricompense 1-di-3), badge di topic e di boss, capitoli in sequenza con prerequisiti.
- **Fictional:** narrativa dei quattro boss e del dungeon-roadmap.
- **Social:** classifica XP dell'Hub con riga "tu" evidenziata e card profilo con punteggio, livello e badge remoti (solo lettura, con stati offline); nessuna leaderboard locale né condivisione social (tagliate per scope).

3.2 Lente Octalysis

Lettura secondo Chou [3]: il nucleo è **Sviluppo/Realizzazione (CD2)** nella soglia 80 % e nei livelli, **Creatività/Feedback (CD3)** nella scelta 1-di-3 e nel deck, **Possesso (CD4)** nelle carte e nei badge, **Imprevedibilità (CD7)** nella pesca boss dai topic del capitolo, **Perdita (CD8)** nel rischio boss (retry senza XP). Significato/Narrativa (CD1) è coperto dalla lore dei boss e dalle intro di capitolo; Influenza sociale (CD5) è medio via Hub (classifica + card) pur senza social locali; Scarsità (CD6) è medio-bassa via boss-gate e daily reward una-tantum (visualizzazione dei pesi in Fig. 3).

Chiuso: voci A-E implementate e verificate. A: intro per capitolo + lore boss pre-fight + finale con statistiche; B: titoli di capitolo + avatar persistito; C: soglie e mosse distintive per boss (Sentinel 50/25, Leviathan 60/30, Archon 75/50/25, Warlord); D: classifica XP Hub + card profilo (punteggio, livello, badge); E: bonus XP giornaliero una-tantum. Suite verde, si veda il Cap. 6.

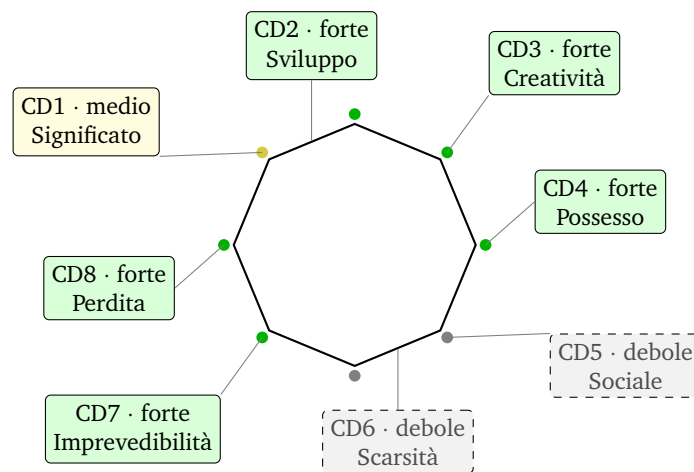


Figura 3: Ottagono Octalysis: pesi dei Core Drive nel progetto (dettagli nel testo).

Capitolo 4

Aesthetics e Dynamics

Estetica fantasy-dungeon sobria: la roadmap è la mappa del dungeon, i topic sono stanze da conquistare, i boss sono i guardiani di capitolo con nome e mosse propri (Syntax Sentinel, Logic Leviathan, Abstraction Archon, Widget Warlord). Le rarità delle carte usano codice colore; le animazioni sono volutamente sobrie per non distrarre dallo studio.

Dynamics: la progressione è doppia ma semplice – **mastery** (XP, livelli, badge, sblocco nodi) e **potere** (deck più forte tramite ricompense 1-di-3), così che studiare meglio significhi combattere meglio. La soglia 80 % mantiene il flow senza frustrare (tentativi illimitati ma penalizzati); il boss a 10 HP contro 50 HP del giocatore con quiz-danno a fine turno crea archi di tensione brevi e leggibili. Nessun dato di esperienza utente è ancora raccolto: si veda il Cap. 7. Riferimenti di design: *Duolingo* (quiz brevi, streak, soglie chiare) per il lato studio e *Slay the Spire* (deck-building, energia per turno, boss a fasi) per il lato combattimento.

4.1 Struttura dei capitoli (esempio)

Capitolo	Boss	Dominio	Meccanica distintiva
1 – Dart Basics	Syntax Sentinel	Tipi, operatori, null safety	Type Error; enrage 50/25 %
2 – Control & Data	Logic Leviathan	Flow, funzioni, collezioni	Infinite Loop; doppia mossa <60 %
3 – OOP & Async	Abstraction Archon	Classi, async	Override ignora Block; soglie 75/50/25
4 – Flutter Basics	Widget Warlord	Widget, layout, stato	SetState Crash; fight a 10 HP

Tabella 4.1: Capitoli, boss e passive uniche.

4.2 Carte evolutive (esempio)

Rarità	Esempio	Danno	Effetto bonus
Comune (7)	Strike	6	Nessuno
Rara (5)	Poison Strike	7	Poison ×3
Epica (5)	Critical Strike	12	Critico 50 % ×2
Leggendaria (4)	Apocalypse	25	Poison/Burn/Bleed ×10

Tabella 4.2: Rarità reali delle 21 carte (+10 knowledge epiche = 31 totali; costo in energia 0–4).

4.3 Dinamiche positive

Chi studia con costanza accumula deck forti e sale di livello prima dei boss; chi ragiona sul boss (passive in Tab. 4.1) ottimizza il deck e vince con meno tentativi; la scelta 1-di-3 abitua a valutare trade-off rischio/ricompensa.

4.4 Dinamiche negative e mitigazioni

- **Quiz anxiety:** la soglia 80 % può spaventare – mitigata da tentativi illimitati, hint (explanation) dopo errori ripetuti e assenza di timer;
- **Stress da classifica:** mitigato da leaderboard solo remota e opzionale, con enfasi su progressione individuale e stati offline chiari;
- **Frustrazione da sconfitta boss:** mitigata da retry a HP pieni senza perdita di progressi (solo senza XP).

4.5 Personalizzazione

Titoli di capitolo, avatar con cornici e deck costruito dalle proprie scelte: il profilo racconta come giochi, non solo quanto hai studiato.

4.6 Adattabilità

I quiz del boss sono pescati dai topic del capitolo (più hai studiato, più domande riconosci); topic falliti tornano in evidenza come ripasso. La difficoltà resta fissa per scelta (niente scaling nascosto): la variabilità viene da pesca, passive ed enrage.

Capitolo 5

Architettura e scelte tecniche

App Flutter offline-first. L'architettura è riassunta in Fig. 4; per i dettagli implementativi si rimanda al documento gemello `specifica.pdf`.

5.1 Layer

- **UI + Provider:** schermate (home, roadmap, topic, quiz, ricompensa, boss) con view-model Provider; un'unica fonte di verità locale per il progresso (PlayerProgress: XP, livello, vite, streak, deck, sblocchi).
- **Repository:** sei repository con responsabilità singola — roadmap (4 capitoli, 29 nodi, prerequisiti a catena tramite `connections`), testi dei topic, quiz (11 da 2–5 domande), boss (nomi, mosse, soglie, quiz del capitolo), ricompense (scelta 1-di-3, limite 1/topic) e persistenza (save per utente `Savev1` su `SharedPreferences`).
- **Engine:** logica di combattimento (HP 10 vs 3, energia 3, quiz-danno) e regole di sblocco, XP, streak e livelli isolate dai widget.
- **Hub client con fallback:** `EngineClient` reale e `FakeEngineClient` offline. Senza credenziali a runtime non parte alcuna chiamata di rete e l'app è identica (offline-first).

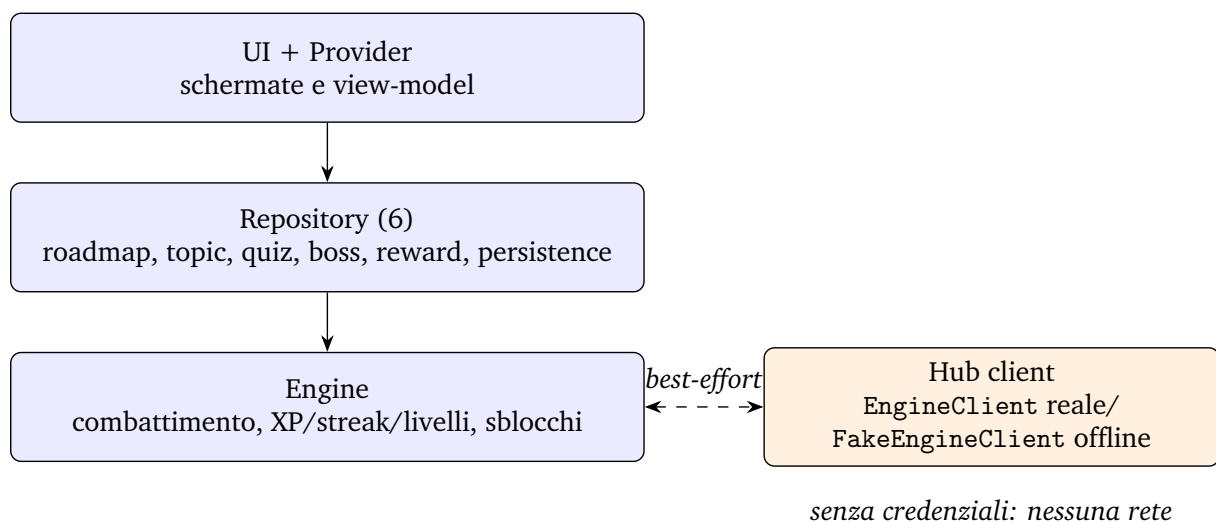


Figura 4: Schema a blocchi: UI/Provider e Repository appoggiano sull'Engine locale; l'Hub (reale o fake) è solo uno specchio best-effort di XP, badge e livelli, con fallback offline totale.

5.2 Integrazione Hub (contratto)

Game **Slay The Code** su console Hub (gameId = 6a9dbf66cc89679981fe7893, owner slay), API `gamification-api.createlab-univaq.it/api/v1` [1]. Risorse: action `quiz_completed`, `claim_reward`, `boss_defeated`; Point `xp`; Level `experience` su `xp` (L1 0, L2 100, L3 500); collezione badge `slay_badges`; rule `quiz_xp`, `topic_badge`, `boss_badge`. Eventi inviati `best-effort` dai view-model di `quiz`, `reward` e `boss`, solo a quiz superato (soglia 80% verificata in app), con data sempre presente in `snake_case` (es. `xp_amount`, `badge`). Solo le 3 action di contratto partono (`allowlist`); gli altri eventi locali non vengono inviati. Il `claim_reward` parte anche dal `pick-1-of-3` e il `boss_defeated` parte sempre, anche senza nodo `roadmap`. Autenticazione a runtime via `--dart-define` (Bearer 24h), credenziali mai nel repo; `playerId` stabile tipo `slay_<uuid>`, uno per utente, salvato in `SharedPreferences`.

5.3 Scelte tecniche

Flutter + Provider per semplicità e testabilità; persistenza locale su `SharedPreferences` con **save per utente** e `auth` con hash salato (nessun backend proprio); Hub come unico servizio remoto, usato solo come specchio di XP/badge/livelli con `fallback` totale. Ogni regola di `gamification` corrisponde a una action Hub, così che il `GamiDOC` resti fonte di verità dei parametri (soglie, danni, HP, costi).

Capitolo 6

Stato implementazione (onesto)

6.1 Cosa gira davvero

App Flutter offline-first: roadmap di 4 capitoli e 29 nodi con 11 quiz da 2–5 domande (soglia 80 %; Inheritance e Async senza quiz: fix pianificato), reward 1-di-3 per tipo con limite 1/topic, boss 10 HP / giocatore 50 HP / energia 3 con quiz-danno, vite 3, XP/livelli 0/100/500 (quiz +100, boss +100, come l'Hub), save per utente con auth locale, login/logout, classi iniziali con deck differenziati, titoli + avatar, soglie e mosse distintive per boss, daily reward +25 XP una-tantum, narrativa intro/finale, classifica e card Hub, New Run con conferma e wipe, animazioni sobrie. Hub: eventi `quiz_completed/claim_reward/boss_defeated` best-effort con XP/badge/livelli remoti e fallback offline completo.

6.2 Tagli dichiarati (future work)

Dichiarati nel piano (§2) e NON implementati: shop/coins, condivisione social, leaderboard locale (c'è quella Hub), signup remota, roadmap extra, achievement avanzati oltre badge topic/boss, eventi testuali random, branching di campagna, difficoltà adattiva oltre la pesca boss, monthly quest, animazioni extra. Voci mockup M1 Sign-in e M2 Option2: rimosse o disattivate. Semplificazioni dichiarate: preview reward solo descrittiva; vittoria boss assegna tutte le reward del nodo; override manuale “Done” con XP lasciato come scelta esplicita.

Chiuso. Voci B–E implementate e verificate (test verdi): titoli + avatar, passive boss, leaderboard e card Hub, daily reward. La voce A (narrativa) è implementata: intro capitoli, lore pre-fight e finale con statistiche. Nessun comportamento futuro è descritto al presente.

Capitolo 7

Valutazione, KPI e Analytics

7.1 Valutazione (protocollo da eseguire)

Think-aloud + questionari (SUS, IMI, nucleo GEQ) su 5 utenti, sessioni da circa 25 min sui task T1–T7 (avvio, topic, quiz 80 %, reward, boss, isolamento utenti, coerenza XP HUD/Hub). Dettagli operativi in docs/assignment/04_evaluation.md. Sezioni *metodo* / *profilo partecipanti* / *risultati* / *riflessioni* da compilare dopo le sessioni.

DA COMPLETARE (User Evaluation da svolgere). Nessuna sessione è stata ancora svolta: nessun risultato utente viene riportato in questo documento. Compilare metodo, profili, risultati e riflessioni solo dopo le sessioni reali.

7.2 KPI e Analytics

KPI candidati: retention D1/D7, tasso di mastery al primo tentativo (soglia 80 %), tempo medio per boss, carte per vittoria, coerenza XP percepita HUD/Hub (task T7), $SUS \geq 70$. Eventi tracciabili via Hub (data nelle executions): `quiz_completed`, `claim_reward`, `boss_defeated`. Le soglie di successo saranno fissate dopo la prima tornata di dati reali.

Bibliografia

- [1] Gamification Hub Univaq, console <https://gamification-webapp.createlab-univaq.it> e API <https://gamification-api.createlab-univaq.it/api/v1> (contratto eventi in docs/assignment/hub_setup.md).
- [2] A. Molina et al., materiali GamiDOC e user research (think-aloud + questionari), corso EGS.
- [3] Y.-K. Chou, *Actionable Gamification*, Octalysis Media, 2015.
- [4] A. M. Toda et al., “A taxonomy of game elements for gamified applications”, *SIGCSE*, 2019.